

Article

BPMN-Based Design of Multi-Agent Systems: Personalized Language Learning Workflow Automation with RAG-Enhanced Knowledge Access [†]

Hedi Tebourbi ¹ , Sana Nouzri ^{1,*} , Yazan Mualla ^{2,*} , Meryem El Fatimi ³ , Amro Najjar ⁴ ,
Abdeljalil Abbas-Turki ²  and Mahjoub Dridi ² 

¹ Faculty of Science, Technology and Medicine, University of Luxembourg, Belval Campus, 2 Place de l'Université, L-4365 Esch-sur-Alzette, Luxembourg

² Faculty of Computer Science, Université de Technologie de Belfort-Montbéliard (UTBM), CIAD UR 7533, F-90010 Belfort, France; abdeljalil.abbas-turki@utbm.fr (A.A.-T.); mahjoub.dridi@utbm.fr (M.D.)

³ Department of Computer Science, Faculty of Science Semlalia, Cadi Ayyad University, Bd Abdelkrim Al Khattabi, Marrakech 40000, Morocco; meryem.elfatimi@uir.ac.ma

⁴ Luxembourg Institute of Science and Technology, L-4362 Esch-sur-Alzette, Luxembourg; amro.najjar@list.lu

* Correspondence: sana.nouzri@uni.lu (S.N.); yazan.mualla@utbm.fr (Y.M.)

[†] This article is a revised and expanded version of a paper entitled "Beyond Chatbots: Enhancing Luxembourgish Language Learning Through Multi-agent Systems and Large Language Model", which was presented at the 25th International Conference of Principles and Practice of Multi-Agent Systems, Kyoto, Japan, 18–24 November 2024. Published in Lecture Notes in Computer Science, Vol. 15395; by Springer Nature: Cham, Switzerland, 2025, and is Available online at: https://doi.org/10.1007/978-3-031-77367-9_29.

Abstract

The intersection of Artificial Intelligence (AI) and education is revolutionizing learning and teaching in this digital era, with Generative AI and large language models (LLMs) providing even greater possibilities for the future. The digital transformation of language education demands innovative approaches that combine pedagogical rigor with explainable AI (XAI) principles, particularly for low-resource languages. This paper presents a novel methodology that integrates Business Process Model and Notation (BPMN) with Multi-Agent Systems (MAS) to create transparent, workflow-driven language tutors. Our approach uniquely embeds XAI through three mechanisms: (1) BPMN's visual formalism that makes agent decision-making auditable, (2) Retrieval-Augmented Generation (RAG) with verifiable knowledge provenance from textbooks of the National Institute of Languages of Luxembourg, and (3) human-in-the-loop validation of both content and pedagogical sequencing. To ensure realism in learner interaction, we integrate speech-to-text and text-to-speech technologies, creating an immersive, human-like learning environment. The system simulates intelligent tutoring through agents' collaboration and dynamic adaptation to learner progress. We demonstrate this framework through a Luxembourgish language learning platform where specialized agents (Conversational, Reading, Listening, QA, and Grammar) operate within BPMN-modeled workflows. The system achieves high response faithfulness (0.82) and relevance (0.85) according to RAGA metrics, while speech integration using Whisper STT and Coqui TTS enables immersive practice. Evaluation with learners showed 85.8% satisfaction with contextual responses and 71.4% engagement rates, confirming the effectiveness of our process-driven approach. This work advances AI-powered language education by showing how formal process modeling can create pedagogically coherent and explainable tutoring systems. The architecture's modularity supports extension to other low-resource languages while maintaining the transparency critical for educational trust. Future work will expand curriculum coverage and develop teacher-facing dashboards to further improve explainability.



Academic Editor: Tudor Groza

Received: 12 July 2025

Revised: 23 August 2025

Accepted: 6 September 2025

Published: 17 September 2025

Citation: Tebourbi, H.; Nouzri, S.; Mualla, Y.; El Fatimi, M.; Najjar, A.; Abbas-Turki, A.; Dridi, M. BPMN-Based Design of Multi-Agent Systems: Personalized Language Learning Workflow Automation with RAG-Enhanced Knowledge Access. *Information* **2025**, *16*, 809. <https://doi.org/10.3390/info16090809>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

Keywords: MAS; language learning; BPMN; RAG; low-resource languages; Luxembourgish; speech recognition; educational pedagogy

1. Introduction

Recent advances in conversational Artificial Intelligence (AI), particularly through systems like ChatGPT 4o, have demonstrated the potential for AI-driven language learning [1]. However, current AI-driven language learning applications face significant limitations: they often rely on single-agent architectures that struggle to manage the complexity of comprehensive language education, and they frequently produce content errors or “hallucination” that undermine pedagogical effectiveness [2–4].

These challenges are particularly acute for low-resource languages like Luxembourgish, where training data scarcity exacerbates model limitations. With Luxembourgish being the main language of only 48.9% of Luxembourg’s population as of 2021, down from 55.8% in 2011, and spoken by approximately 275,000 people as their primary language [5,6], traditional large language models (LLMs) often fail to generate accurate, contextually appropriate educational material [7]. Three fundamental challenges persist in AI-driven language education for low-resource contexts: First, the scarcity of training data exacerbates LLMs’ tendency toward hallucinations [8]. Second, most systems lack pedagogical structure, focusing narrowly on vocabulary drills while neglecting the interleaving of reading, writing, listening, and speaking skills shown to maximize retention [9,10]. Third, cultural specificity in language usage (e.g., Luxembourgish code-switching with French/German) requires careful modeling that monolithic architectures cannot provide [11]. These limitations collectively hinder the development of comprehensive, reliable AI tutors for marginalized languages. While Luxembourgish serves as our case study, these challenges are shared by many other low-resource languages, such as Welsh, Basque, and Maltese, making the problem space relevant beyond the Luxembourgish context.

Furthermore, current language learning applications utilizing LLMs predominantly focus on vocabulary acquisition through role-playing conversations and providing immediate feedback derived from model-generated content [12]. This method, however, can result in exposure to inaccuracies or model “hallucinations” and fails to address the comprehensive nature of the language learning process, which extends beyond conversational practice to include reading comprehension, listening skills, and grammatical competency [13]. Effective language acquisition necessitates robust pedagogy, efficient teaching methodologies, reliable content, and a supportive teacher-student dynamic [10]. While Multi-Agent Systems (MAS) have shown promise in educational contexts through distributed task management [14,15], their application to language learning for low-resource languages remains underexplored. In addition, few existing approaches formalize instructional logic in a transparent, reproducible way that can be independently implemented without reliance on a single proprietary stack.

Based on our previous works [16,17], this article addresses these challenges through a novel integration of three key technologies: Business Process Model and Notation (BPMN) for workflow transparency and specification [18], MAS for distributed task execution, and Retrieval-Augmented Generation (RAG) for auditable knowledge access and enhanced content accuracy [19]. By modeling language learning as a structured business process, where BPMN diagrams explicitly define pedagogical workflows that are then executed by specialized agents, each focusing on specific learning aspects (conversation, reading, listening, grammar), we ensure a comprehensive learning experience that addresses the holistic nature of language acquisition. By grounding agent responses in a RAG-enhanced

knowledge base built from validated educational materials from the National Institute of Languages of Luxembourg (INL), we significantly reduce content errors while maintaining pedagogical coherence. Because this methodology is language-agnostic and resource-adaptive, it can be scaled to other low-resource or domain-specific learning contexts by replacing the knowledge base and adjusting BPMN workflows.

Explainable Artificial Intelligence (XAI) is a field that aims to provide techniques, models, and methods for developing XAI-based systems [20–23]. These systems enable users and other human actors to better understand AI’s decision-making, which, in turn, can improve factors such as understandability, trust, and transparency [24–27], particularly in data-driven AI [28–30].

Our approach extends beyond traditional Intelligent Tutoring Systems (ITS) [31] by implementing dynamic agent collaboration rather than rule-based responses, enabling more adaptive and personalized learning experiences while prioritizing explainability through BPMN’s transparent workflow modeling. By formalizing agent interactions and decision logic in BPMN diagrams, we provide educators and learners with auditable, human-interpretable insights into AI-driven pedagogical choices (e.g., activity sequencing, error correction pathways). This explainability addresses the “black-box” limitations of conventional LLMs, aligning with XAI principles of trust and accountability in educational AI. The integration of Speech-To-Text (STT) and Text-To-Speech (TTS) technologies further enhances the immersive learning environment, addressing the conversational practice needs critical for language proficiency development. Furthermore, the opaque decision-making of LLMs hinders pedagogical trust. Our framework inherently addresses this by: (i) decomposing complex workflows into auditable agent tasks, (ii) grounding responses in traceable knowledge sources, and (iii) enabling human-in-the-loop validation of XAI for educational systems.

This work is guided by three research questions: (1) Can BPMN-modeled, MAS-executed workflows improve reliability and pedagogical structure in AI-driven language learning compared with single-agent baselines? (2) Does RAG integration measurably reduce hallucinations and improve learner engagement in low-resource contexts? (3) Can the proposed architecture be generalized to other low-resource languages or domain-specific instructional scenarios?

The main contributions of this paper are as follows:

- A novel BPMN-to-MAS transformation methodology that converts pedagogical workflows into executable MAS, bridging formal process modeling with AI-driven education.
- Integration of RAG technology to ensure accurate, contextually grounded language instruction while mitigating LLM hallucinations.
- Implementation of a complete Luxembourgish learning platform (A1–B2) with React/FastAPI frontend, LangGraph core, ChromaDB vector store, and STT/TTS pipelines.
- Empirical evaluation showing strong response accuracy (RAGAs: Context Relevancy = 0.87, Faithfulness = 0.82, Answer Relevancy = 0.85) and high learner satisfaction in a pilot (85.8% ease-of-use, 71.4% engagement).
- A generalizable framework for low-resource language education that combines formal process modeling, distributed AI agents, and knowledge-grounded generation.

2. Related Work

2.1. BPMN for Educational Process Modeling

BPMN has been widely adopted to formalize and communicate complex workflows in education. García-López et al. demonstrated how BPMN diagrams capture pedagogical sequences—such as lesson plans and assessment flows—improving collaboration between instructors and system designers [32]. Similarly, Costa and Silva applied BPMN to model

adaptive e-learning paths, enabling conditional branching based on learner performance while improving course reusability and personalization [33]. Nevertheless, these efforts typically stop at static documentation and do not produce executable, adaptive process definitions that integrate with AI agents at run time. Advances in modeling human–agent collaborative workflows [34] and adaptive BPMN-based learning flows [35] show promise but remain at the conceptual or prototype stage. Furthermore, none of these BPMN implementations have been operationalized in a way that directly supports reproducibility or language portability, leaving a gap for frameworks that can compile pedagogical process models into executable AI-driven tutoring systems across domains. Our work operationalizes BPMN diagrams into a live MAS execution layer integrated with RAG for knowledge grounding, which, to our knowledge, has not been reported in BPMN-based educational systems.

2.2. Multi-Agent Systems in Education

Intelligent tutoring systems have long leveraged MAS to distribute educational tasks among specialized agents. Wooldridge and Jennings’s foundational survey [36] laid out the theory of agent collaboration, and Ivanova et al. [37] later proposed an agent-oriented architecture for strategy-based e-learning with distinct agents handling content delivery, assessment, and learner tracking. More recent LLM-based MAS frameworks—such as AutoGen [38] and LangGraph [39]—provide generic scaffolding for multi-agent orchestration but have not been tailored to formal pedagogical workflows or low-resource language settings. LLM tutoring systems such as AutoTutor-LLM [40] implement guardrails and pedagogy-aware prompts, but they do not couple MAS orchestration with explicit, BPMN-defined instructional logic. There is limited evidence of MAS being applied in combination with explicit BPMN-defined instructional logic, which is central to our first research question on improving pedagogical structure.

2.3. Retrieval-Augmented Generation

RAG combines neural generation with external retrieval to ground outputs in factual documents. Lewis et al. [19] first showed that RAG significantly reduces hallucinations in knowledge-intensive NLP tasks. In education, Wang et al. [41] applied RAG to AI tutoring systems and reported a 40% reduction in factual errors and a 25% increase in answer relevance. Recent studies [42,43] reaffirm RAG’s effectiveness in reducing hallucination, yet existing RAG-based solutions do not incorporate formal process models or MAS. No prior study, to our knowledge, has systematically evaluated whether RAG can mitigate hallucinations when embedded in a BPMN-to-MAS pipeline for low-resource language learning, directly addressing our second research question.

2.4. LLM-Powered Language Learning Chatbots

Generative AI chatbots like ChatGPT have become popular for language practice. Belda-Medina and Calvo-Ferrer [1] showed that LLM conversational partners can boost vocabulary retention. Cavojský [12] integrated GPT-3 in classroom settings, yielding fluency gains, but also observed persistent hallucinations. Huang et al. [2] provide a taxonomy of these hallucinations, and Rzepka et al. [44] analyzed their impact on learner trust. Commercial platforms (e.g., Duolingo Max [45]) now harness GPT-4, yet still emphasize gamified vocabulary gains over a comprehensive, skills-based curriculum. Latest deployments [46] stress integrating conversation with other core skills such as reading, listening, and grammar. This underscores the lack of integrated solutions that balance conversation with other skills such as reading, listening, and grammar in a structured, explainable sequence, one of the key design goals of our framework.

2.5. Technologies for Low-Resource Languages

Low-resource languages like Luxembourgish face acute data scarcity. Lavergne et al. [47] surveyed existing Luxembourgish NLP tools and found major gaps in corpora and models. Multilingual transformers (XLM-R [48], mBERT [49]) afford zero-shot capabilities but often underperform in specialized domains. For speech, Whisper-based STT fine-tuned on RTL.lu achieves WERs of 18–28% [8], yet has seen little application in structured educational systems. Work on Luxembourgish NLP is advancing with extended resources [50–53], but these efforts focus on resource creation rather than integration into pedagogically structured AI tutoring systems. Comparable challenges are faced by other low-resource languages such as Basque, Maltese, or Welsh, highlighting the importance of designing architectures that are resource-adaptive and language-agnostic, as posed in our third research question.

2.6. Research Gap

While prior work has independently explored BPMN for high-level educational design [32], MAS for distributed tutoring [36,37], RAG for grounding LLM outputs [19,41], LLM-based chatbots for language practice [1,12], and low-resource language models [8,47], no existing system brings all of these together into a cohesive, executable framework that performs the following:

1. Uses BPMN to specify pedagogical workflows.
2. Orchestrates specialized LLM-powered agents via MAS.
3. Grounds all content in vetted external knowledge with RAG.
4. Incorporates real-time voice interaction (STT/TTS) for a low-resource language.

Our architecture unites these components into a reproducible, language-agnostic pipeline adaptable to other low-resource or domain-specific contexts simply by replacing the knowledge base and BPMN models. This positions our work beyond current systems in both methodological scope and practical applicability.

3. System Design and Architecture

Our system design is built around the integration of three core components—BPMN, MAS, and RAG—which together enable a modular, intelligent, and process-driven learning environment. While the current deployment uses a combination of proprietary (e.g., GPT-4-turbo, LangSmith) and open-source components (e.g., FastAPI, ChromaDB, Whisper), the architecture is deliberately designed to be technology-agnostic. Any LLM, vector store, or observability tool can be substituted, enabling reproducibility and portability across institutional or national contexts.

- **BPMN for Workflow Modeling:** BPMN is used to define and structure the high-level learning workflow, where each task represents a discrete, pedagogically motivated educational activity (e.g., “Grammar Practice” or “Listening Drill”). This visual modeling ensures that the learning journey is transparent, auditable, and adaptable. For example, a gateway in the BPMN diagram may check a learner’s comprehension score before deciding whether to proceed to the next module or trigger a remedial loop. This workflow specification is language-agnostic: replacing the underlying content source (e.g., Luxembourgish INL materials) with another language corpus automatically adapts the curriculum while preserving the pedagogical logic.
- **MAS for Process Execution:** The BPMN workflow is executed by a Multi-Agent System where each agent is domain-specific and optimized for a specific pedagogical role. Instead of a single, monolithic LLM, specialized agents (Conversational, Grammar, Reading, Listening) operate in parallel and exchange structured messages. This allows the platform to replicate the modularity of Luxembourgish textbooks, in which

activities are clearly separated by skill type, and to allocate computational resources more efficiently. In our current implementation, MAS orchestration is built specifically on LangGraph to leverage its graph-based state management and node orchestration capabilities. While the BPMN definitions and pedagogical design could be reused, the execution layer is optimized for LangGraph's architecture.

- **RAG for Knowledge Grounding:** Each agent is connected to a Retrieval-Augmented Generation pipeline that provides access to vetted educational content. Using vector stores built from OCR-processed INL textbooks, the agents retrieve semantically relevant chunks to anchor their output in authoritative material. This prevents hallucinations, aligns answers with CEFR-level objectives, and maintains consistency across sessions. While our pilot uses INL Luxembourgish resources, the RAG pipeline can be rebuilt from any approved corpus in another language or subject area, making the approach applicable well beyond Luxembourgish.

The architecture (Figure 1) draws inspiration from the pedagogical segmentation in INL textbooks, which organize activities into types such as Schwätzt (Speak up), Lauschtert (Listen), Liest (Read), Kombiniert (Combine), Notéiert/Schreift (Note/Write), Kräizt un (Cross on), and Ënnersträichen (Underline). This structure informs our modularity: each activity type is mapped to a specialized agent or sequence of agents, ensuring that all four core language skills are addressed in a balanced, CEFR-aligned curriculum. These mappings can be redefined for any instructional program that has skill-segmented activities, allowing the same MAS + BPMN + RAG pipeline to be used in other low-resource languages or even in non-language domains such as technical training.

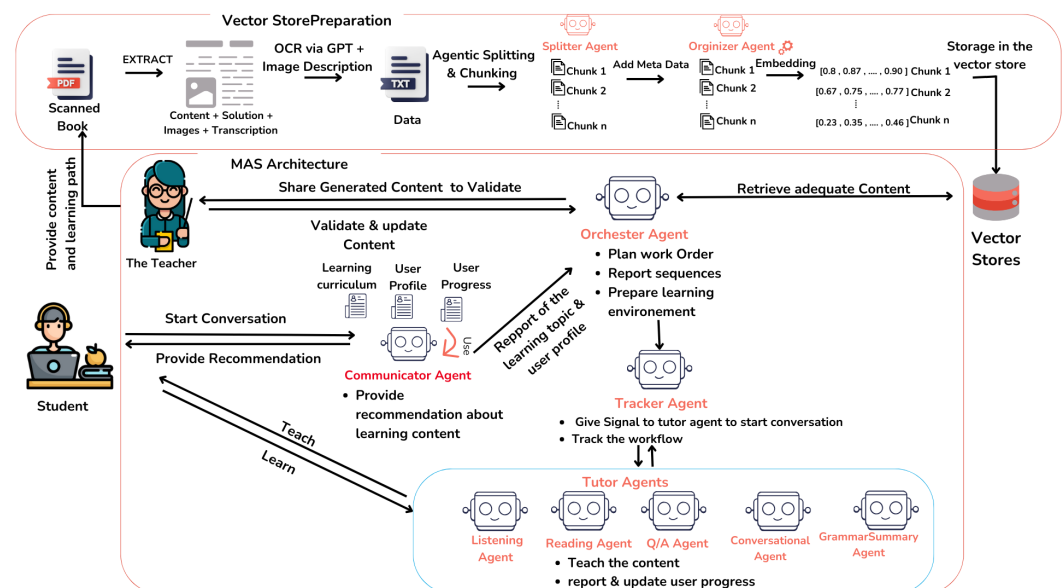


Figure 1. Collaborative learning via agent communication and RAG-based MAS for language learning [17].

3.1. Description of the Architecture Workflow

Figure 1 illustrates the complete architecture, which can be divided into two major phases: (i) vector store preparation and (ii) MAS execution for teaching and learning.

1. **Vector Store Preparation (top of the diagram):** The first step occurs before interaction with learners. Content is extracted from scanned INL textbooks using OCR via ChatGPT Vision, which outputs transcribed text and descriptions of embedded images. The Splitter Agent then divides this material into semantically coherent chunks, while the Organizer Agent enriches each chunk with metadata (Thema, activity type,

intended tutor agent). The annotated chunks are embedded into dense vectors and stored in the vector database. This repository constitutes the knowledge base for Retrieval-Augmented Generation (RAG).

2. MAS Architecture (bottom of the diagram): During live sessions, the Communicator Agent collects the learner profile (level, past progress, preferences) and provides recommendations on relevant learning content. This information is passed to the Orchestrator Agent, which validates retrieved content with the teacher, plans the work order, prepares the learning environment, and sequences activities. The orchestrator then calls the Tracker Agent, whose role is to ensure correct execution order, signal the appropriate tutor agent, and monitor workflow progression. Each Tutor Agent (Conversational, Reading, Listening, Q&A, Grammar Summary) receives the selected content chunk as input, delivers the activity to the learner, and outputs updated progress and performance indicators, which are reintegrated into the learner profile.
3. Inputs and Outputs: For clarity, inputs and outputs are now described explicitly:
 - The input of the Vector Store Preparation pipeline is raw textbook content (text + images); its output is an embedded, metadata-enriched vector store.
 - The input of the Communicator Agent is the learner profile; its output is a recommendation about learning content.
 - The input of the Orchestrator Agent is the learner profile and validated content; its output is an ordered pedagogical sequence.
 - The input of the Tracker Agent is the orchestrator's plan; its output is signals to tutor agents and execution logs.
 - The input of each Tutor Agent is a content chunk; its output is updated learner progress and performance data.

To further increase clarity, detailed workflows for each major agent (Communicator, Orchestrator, Tracker, Tutor) are provided as BPMN Activity diagrams in Appendix A, complementing the general overview shown in Figure 1.

3.2. Core Agent Roles and Responsibilities

3.2.1. Core Agents

- (A) Communicator Agent
Acts as the system's primary entry point for the learner. It collects and updates the learner profile (e.g., CEFR level, preferred learning style, past performance), recommends activities tailored to their needs, and establishes the learning path. This agent justifies its role by ensuring that no session begins without a pedagogically relevant and personalized plan. Its logic is independent of language or domain—profiles can store any set of learner metadata, making it adaptable for other subjects or languages by replacing the content retrieval layer.
- (B) Orchestrator Agent
Serves as the "session director," reading the BPMN workflow, retrieving the correct content for each step via RAG, and sequencing activities in pedagogically meaningful ways. Its existence is essential to guarantee that content flows logically (e.g., listening before speaking practice) and that agents receive the exact context they need. Since it consumes BPMN definitions, the same Orchestrator can execute any compliant process model, enabling reuse across institutions or curricula with different content sources.
- (C) Tracker Agent
Ensures that the session is executed in the correct order and that no activity is skipped. It waits for each tutor agent to finish before activating the next one, logs task completion, and maintains timing constraints. This agent exists to maintain

session integrity and support explainability by providing a clear execution trace. The Tracker Agent's state management relies on LangGraph's graph execution model, meaning that preserving identical execution integrity would require using LangGraph or an equivalent graph-based orchestration engine.

(D) Specialized Tutor Agents

These agents map directly to the main skill areas in language learning and replicate textbook activity types. Each has a unique role:

- **Conversational Agent:** Simulates real-life dialogue, conducts role-plays, and evaluates spoken language. It uses STT to assess pronunciation and fluency, re-prompts after repeated pronunciation errors, and expands vocabulary through contextual use. This agent exists to address the speaking component of the CEFR framework.
Its STT/TTS pipeline can be switched to open-source models (e.g., Whisper, Vosk) or commercial APIs in any target language.
- **Reading Agent:** Presents texts, checks comprehension through summaries or questions, and introduces vocabulary in context. It is triggered when BPMN conditions indicate a need for reading comprehension improvement.
Adaptation to other languages simply requires a corpus with aligned reading materials and CEFR-like difficulty labels.
- **Listening Agent:** Plays audio excerpts, transcribes learner responses, and provides corrective feedback on recognition errors. It repeats audio when comprehension thresholds are not met.
This agent supports any audio format and can be integrated with multilingual STT engines to extend coverage beyond Luxembourgish.
- **QA Agent:** Presents exercises (fill-in-the-blank, multiple-choice) modeled after "Kombinéiert!" and "Notéiert!/Schreift!" activities. It provides targeted hints after wrong answers and re-tests the learner before moving on.
Exercise templates are generic and can be automatically populated from different datasets, enabling domain transfer.
- **Grammar Summary Agent:** Explains grammar rules, answers learner questions, and runs short practice drills. This agent is called after BPMN gateways detect consistent grammar errors in conversation or writing exercises.
Grammar explanations are sourced from the RAG knowledge base, allowing immediate substitution of language-specific rules for other target languages.

(E) Human Tutor

A teacher will be involved to ensure the linguistic accuracy and coherence of the learning materials. This includes verifying the alignment between text content, audio transcriptions, and visual illustrations. The teacher also validates the relevance and pedagogical sequencing of the content retrieved by the Orchestrator, ensuring it is appropriate for the user's proficiency level and learning objectives. Beyond quality control, the human tutor serves as a critical human-in-the-loop validation layer: they review AI-generated outputs for both correctness and cultural appropriateness, confirm that exercises are aligned with CEFR progression goals, and provide feedback for refining agent prompts and workflow logic. This role ensures that automated decisions remain pedagogically sound, maintain learner trust, and adapt appropriately to edge cases where AI recommendations may not fit the learner's individual needs. In deployments outside Luxembourgish, this role can be filled by subject matter experts in the relevant language or domain, making the validation process universally applicable.

3.2.2. Knowledge Provenance

Every RAG-augmented response in our system is transparently linked back to its original textbook source via the metadata produced by the Organizer agent (see Figure 2). This step is critical for mitigating hallucinations and maintaining pedagogical trust. Prior to storage in the vector database, each chunk is annotated with the following fields—extracted verbatim from the INL A1 materials and cross-checked against the scanned PDF pages to ensure exact provenance:

- Thema (topic title, e.g., “Wéi heescht Dir?”).
- Kategorie (activity type, e.g., “Dialogs and name spelling exercises”).
- Agent (the intended tutor agent for delivery).
- Inhalt (verbatim learning content from the textbook).

These metadata fields are generic and can be adapted for other curricula by changing labels and source mappings, preserving the same provenance-tracking capability across contexts.

```

2  - Kapitel: 1
3  - thema: Moien!... an Addi!
4  - kategorie: Gespréich
5  - agent: conversational agent
6  **Inhalt:**
7  ### Lëtzter Ziler
8  - Moien an Addi soen
9  - Soen a froen, wéi et geet
10 - Sech virstellen an nom Numm vun enger Persoun froen
11 - Soen, vu wou ee kënnt, an eng Persoun duerno froen
12 - Seng Nationalitéit a seng Sprooch(en) nennen
13 - an eng Persoun duerno froen
14
15 - Kapitel: 1
16 - thema: Moien!... an Addi!
17 - kategorie: Gespréich
18 - agent: conversational agent
19 **Inhalt:**
20 ### Conversations
21 1. Person 1: Gudde Mëtteg, Madame!
22   Person 2: Bonjour!
23   illustration : Op dësem Bild gesi mir zwee Leit, déi sech mat engem Handdrock begréissen. Béd soen "Bonjour!" zueinander.
24
25 - Kapitel: 1
26 - thema: Moien!... an Addi!
27 - kategorie: Gespréich
28 - agent: conversational agent
29 **Inhalt:**
30 2. Person 1: Bonjour!
31   Person 2: Bonjour!
32   illustration : Op dëser Illustratioun gesi mir eng Bäckerei-Szene. Eng Bäckereimadam steet hannert dem Comptoir, op deem v
33
34 - Kapitel: 1
35 - thema: Moien!... an Addi!
36 - kategorie: Gespréich
37 - agent: conversational agent
38 **Inhalt:**
39 3. Person 1: Moien Anna!
40   Person 2: Hallo!
41   illustration : Op dësem Bild gesi mir zwee Leit, déi sech begréissen. Eng Persoun, déi e roude a giel gestreiften T-Shirt u
42

```

Figure 2. Organizer Agent output: textbook chunks annotated with pedagogical metadata and source references.

3.2.3. Human Auditing

The platform embeds a human-in-the-loop validation layer that ensures AI-driven learning remains aligned with expert teaching practices. Validators (qualified teachers) systematically review:

- The raw BPMN-defined sequence of activities to confirm the intended pedagogical flow.
- LangSmith logs capturing the exact decision context for each agent’s action.
- The alignment between chunk metadata and delivered exercises, ensuring that each exercise is both relevant and level-appropriate.

This audit process prevents unintended content drift, guarantees that adaptive recommendations follow curriculum guidelines, and maintains consistency between AI output and human educational expertise. Because the BPMN models and logs are stored in open formats, this auditing process can be reproduced in any other implementation, regardless of the underlying LLM or MAS framework.

3.2.4. Workflow Transparency

BPMN diagrams (Figures 3 and A1–A4 in Appendix A) serve as visual documentation of the learning process. They explicitly capture every decision point, gateway condition, and activity branch, for example:

IF pronunciation_error_count > 2 $\rightarrow$ Re-prompt

These visual models allow the following:

- Developers to understand and debug complex agent interactions without needing to interpret raw code.
- Instructors to verify that system behavior aligns with pedagogical best practices.
- Auditors to confirm that decision logic is explicit, deterministic, and traceable.

Since BPMN is an ISO-standard notation, these diagrams remain valid even if the underlying orchestration technology changes, ensuring transparency and reproducibility across deployments.

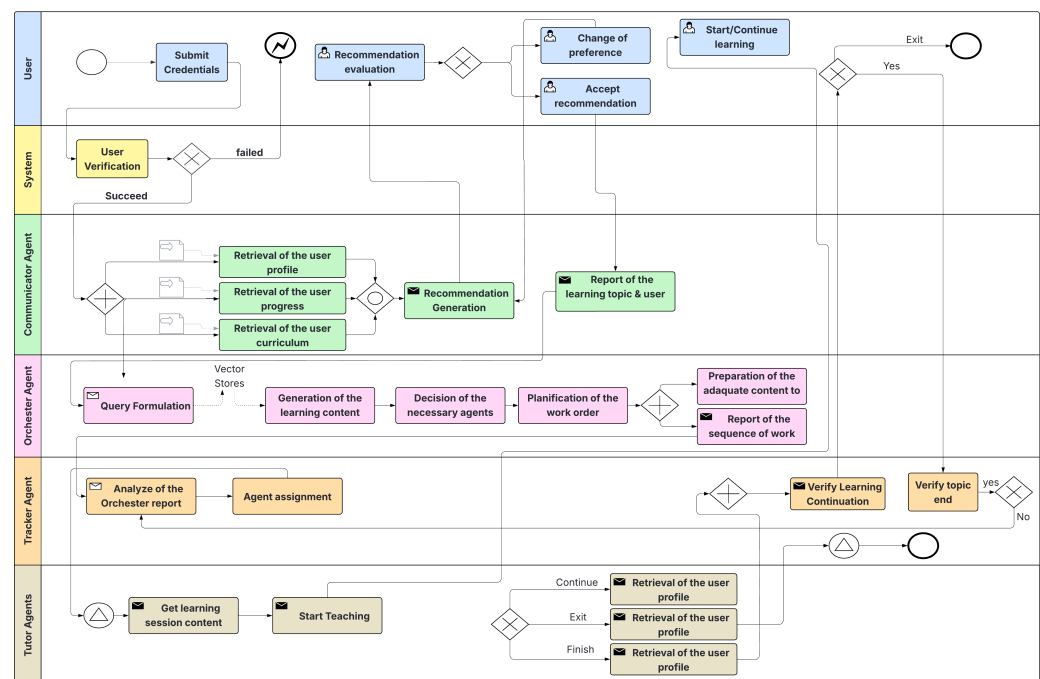


Figure 3. BPMN diagram for the agents interaction [17].

3.3. Explainability Mechanisms

We embed an interactive explainability layer into our multi-agent workflow, ensuring transparency in what would otherwise be a “black-box” system. By integrating LangSmith [54], educators gain granular visibility into the rationale behind each adaptive decision made by the LangGraph engine, creating clear audit trails from learner data to agent actions.

Although our current observability stack uses LangSmith, the same logging architecture can be implemented with open-source alternatives, ensuring explainability is not tied to a single vendor.

LangSmith Role

LangSmith is an interactive observability and debugging platform provided by LangChain Inc. [55]. It captures, stores, and visualizes every prompt, tool call, router

decision, and state change across a LangGraph-driven workflow. As shown in Figure 4, the platform provides a detailed, interactive trace of decision steps.

Through its web-based dashboard, educators and developers can:

- Explore complete prompt histories for each agent node to understand how the model was instructed.
- Inspect intermediate state variables and message payloads to trace decision dependencies.
- Visualize conditional routing paths and loop iterations, making execution flow easy to follow.
- Search and filter on specific student interactions or decision predicates for targeted review.

By integrating LangSmith, our system transforms opaque agent behavior into auditable, human-readable traces, which perform the following:

- Simplifies technical debugging for developers.
- Facilitates pedagogical review for educators.
- Supports compliance and accountability for institutional oversight.

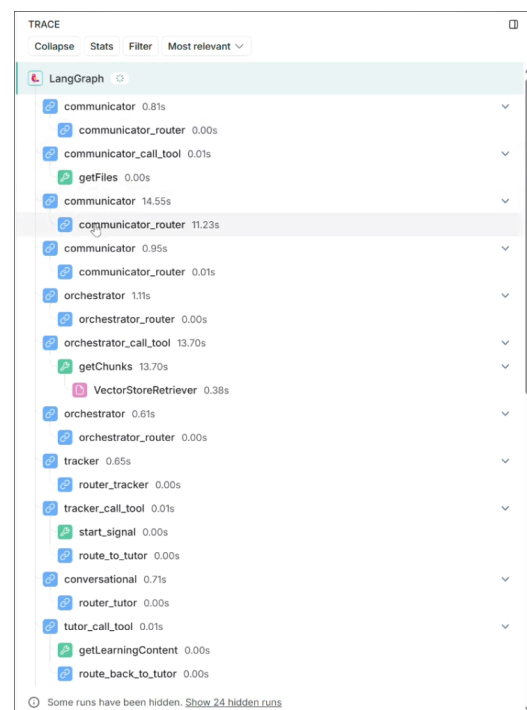


Figure 4. LangSmith trace of decision steps for a sample learner session.

3.4. Pedagogical and Methodological Fit

Our design is grounded in established educational frameworks:

- CEFR and Mastery Learning: BPMN activities map to CEFR skills (speaking, listening, reading, grammar). Gateways implement criterion thresholds (e.g., comprehension or pronunciation scores) to trigger remediation, operationalizing mastery learning.
- CLT/TBLT: The Conversational Agent delivers communicative tasks (role-plays, turn-taking). The QA and Grammar Summary agents provide focus-on-form within meaningful interaction, consistent with communicative and task-based language teaching.

- UDL (Universal Design for Learning): STT/TTS offer multimodal access (input/output). BPMN loops support variable pacing and multiple means of engagement; RAG curates level-appropriate materials for multiple means of representation.
- ADDIE as Executable Workflow: BPMN captures Analyze/Design/Develop/Implement/Evaluate. The Tracker and human-in-the-loop validation close the E (Evaluate) phase with logs and feedback, feeding iterative redesign.
- Evidence-Centered Design (ECD): The Tracker's logs and agent outputs form the evidence model; BPMN tasks define the task model; learner state variables constitute the student model. RAG provenance improves the validity of observed evidence.
- Formative Assessment: QA hints, re-tries, and remedial branches provide continuous formative feedback and data for next-step decisions.

This positioning clarifies how the MAS+BPMN+RAG stack operationalizes well-known pedagogical principles rather than replacing them, and explains the system's role as an explainable, teacher-orchestrated ITS.

4. BPMN to MAS Transformation

4.1. BPMN Modeling of Learning Workflows

To capture how our agents communicate, cooperate, and supervise one another, we represent their interaction flow with a BPMN diagram (see Figure 3). By modeling our MAS in BPMN, we make the system's logic explicit, easily understandable to both technical and non-technical stakeholders, and readily analyzable for correctness and optimization.

4.1.1. Top-Level Orchestration Diagram (Figure 3)

The roles and responsibilities of each pool and lane in the orchestration are summarized in Table 1.

1. Start Event (Message): The User logs in, triggering a message start event in the Communicator lane.
2. User Data Retrieval (Service Tasks): In the Communicator lane, three service tasks are retrieved:
 - UserProfile: personal details and learning objectives.
 - LatestProgressFile: feedback from the previous session.
 - CurriculumOutline: textbook TOC matching the user's proficiency.
3. Personalized Path Generation: A service task builds a LearningPathRecommendation. A message flow delivers it to the User, and an exclusive gateway ("Accept?") loops back for refinement until approval.
4. Query Generation and Dispatch: Once approved, the Communicator constructs a RAGQuery (including topic IDs and proficiency level) and sends it as a message to the Orchestrator.
5. Content Retrieval and Validation: The Orchestrator executes a VectorStoreLookup against ChromaDB, then sends the retrieved material to the Human Teacher for validation (message task) and awaits approval.
6. Workflow Planning: A parallel gateway splits into two branches:
 - Assign each content chunk to its appropriate Tutor Agent.
 - Build the SequenceReport specifying agent invocation order.
 Both branches join before proceeding.
7. Report Emission: Two tasks are called:
 - ContentReport → Tracker (mapping agents to content).
 - SequenceReport → Tracker (ordered list of agents).

8. Tutor Invocation Loop: In the Tracker lane:
 - a. DetermineNextAgent via SequenceReport.
 - b. Send StartSession message to that Tutor Agent.
 - c. Wait (intermediate catch event) for EndSession or EarlyExit.
 - d. Log progress (partial or complete).
 Repeat until no agents remain.
9. End Event: Once all sessions finish, the Tracker emits an end event. The UI displays the updated progress dashboard and may loop back to the Communicator for a new cycle.

Table 1. Pools and Lanes in the Top-Level BPMN Orchestration.

Pool / Lane	Role / Responsibility
User	Human learner interacting via the UI.
Communicator Agent	Retrieves profile data; proposes personalized learning paths; emits RAG queries.
Orchestrator Agent	Fetches and validates content; plans which tutor agents to invoke and in what order.
Tracker Agent	Drives step-by-step activation of tutor agents; logs completion or early-exit signals.
Tutor Agents	Swimlane for specialized tutors (Conversation, Reading, Listening, QA, Grammar Summary).

Pools and lanes correspond to the BPMN swimlanes shown in Figure 3.

To further clarify Figure 3, we justify each workflow action, emphasizing its input, output, and pedagogical role:

- User Credential Submission: Input = learner login data; Output = verification signal. Justified as a prerequisite for linking activities to an individual profile and ensuring continuity of progress tracking.
- Profile, Progress, Curriculum Retrieval: Input = database queries; Output = structured learner state (profile, last progress, curriculum). These steps guarantee that personalization is based on actual learner history rather than generic defaults.
- Learning Path Recommendation: Input = learner state; Output = a personalized path proposal. Justified as the mechanism that maintains learner agency (accept/modify recommendations) while ensuring pedagogical alignment.
- RAG Query Generation: Input = approved recommendation; Output = structured query (topic, level, context). This ensures retrieved material is both content-relevant and level-appropriate.
- Content Retrieval and Validation: Input = RAG query; Output = validated content chunks. Human-in-the-loop validation is critical to prevent propagation of errors or culturally inappropriate material.
- Workflow Planning (Parallel Gateway): Input = validated content; Outputs = (a) tutor-content mapping, (b) execution sequence. This guarantees both correct specialization (each tutor gets the right chunk) and logical sequencing (skills taught in a pedagogically sound order).
- Report Transmission: Input = planning results; Output = content and sequence reports sent to the Tracker. Justified as an explainability feature—each decision is made explicit and logged.
- Tutor Invocation Loop: Input = sequence report; Output = session execution logs. By enforcing strict sequencing and waiting for explicit completion events, this loop prevents skipped steps and enables auditing of learner-agent interactions.

- Session Termination and Feedback: Input = last tutor outputs; Output = updated learner progress and dashboard update. This ensures that every session closes with measurable outcomes and an updated profile for the next cycle.

Together, these justifications highlight how the BPMN diagram goes beyond simple flow visualization: it encodes traceable pedagogical logic where every input and output is explicit, auditable, and grounded in the learner's evolving profile.

4.1.2. BPMN Diagrams for Tutor Agents

Each tutor agent follows a similar BPMN structure of message start, instructional tasks, decision gateways, and message end. We illustrate, as shown in (Figure 5), the Conversation Agent; the Reading, Listening, QA, and Grammar Summary Agents adapt this template with domain-specific tasks.

Example: Conversation Agent (Figure 5)

1. Message Start: Catch StartSession from Tracker.
2. Fetch Content: Load dialogue script and role definitions from ContentReport.
3. Introduction: Outline session goals (e.g., focus on past tense).
4. Role-Play Loop:
 - Prompt user with their first line.
 - Send spoken reply to STT; receive transcription.
 - Gateway G1 (Correct?):
 - If correct, advance to next line.
 - If incorrect, provide corrective feedback and loop back.
 - Repeat until all turns are complete.
5. Wrap-Up: Summarize key vocabulary and structures; write progress fragment.
6. Message End: Send EndSession + progress payload back to Tracker.

The decision logic for gateways in this workflow is summarized in Table 2.

Table 2. Gateway Conditions in the Conversation Agent BPMN Diagram.

Gateway ID	Condition and Action
G1 (Correct?)	IF pronunciation_error_count = 0 → advance to next dialogue turn; ELSE → invoke corrective feedback task and loop back.
G2 (All Turns Completed?)	IF turns_completed = total_turns → proceed to Wrap-Up; ELSE → return to Role-Play Loop.

Defines the branching logic at each decision point in Figure 5.

4.1.3. Overview of Other Tutor Agents

To avoid redundancy, we summarize each agent's core workflow alongside its key gateway conditions:

- Reading Agent (Figure A1): Presents text to read, checks pronunciation via STT, requests a spoken or written summary, evaluates comprehension, teaches new vocabulary, and loops until mastery.
 Gateway R1: IF summary_correct? → continue; ELSE → replay text + re-question.
 Gateway R2: IF comprehension_score > threshold → next activity; ELSE → vocabulary drill.
- Listening Agent (Figure A2): Plays audio clips, prompts learner reproduction, transcribes and evaluates responses, offers vocabulary tips, and loops for reinforcement.
 Gateway L1: IF transcription_accuracy > 80% → next clip; ELSE → replay clip.
 Gateway L2: IF vocab_usage_correct? → continue; ELSE → provide targeted vocabulary drill.

- QA Agent (Figure A3): Displays exercises (fill-in, MCQ), evaluates answers, provides hints on incorrect responses, and summarizes learning goals.
Gateway Q1: IF `answer == key` → correct flow; ELSE → hint task + retry.
Gateway Q2: IF `retry_count > 2` → escalate to Grammar Summary Agent; ELSE → loop for another attempt.
- Grammar Summary Agent (Figure A4): Reviews previous grammar, elicits user questions, explains rules, engages in practice sentences, identifies errors, and closes with a concise rule summary.
Gateway Gs1: IF `user_asks_question` → answer question; ELSE → present practice sentence.
Gateway Gs2: IF `error_count > 3` → trigger additional examples; ELSE → proceed to summary.

By pairing each BPMN activity diagram with a compact gateway-condition table, we provide a crystal-clear mapping from each decision node (“diamond”) to its exact runtime logic. This ensures readers can both visualize the flow and understand the precise branching criteria that drive adaptive learning.

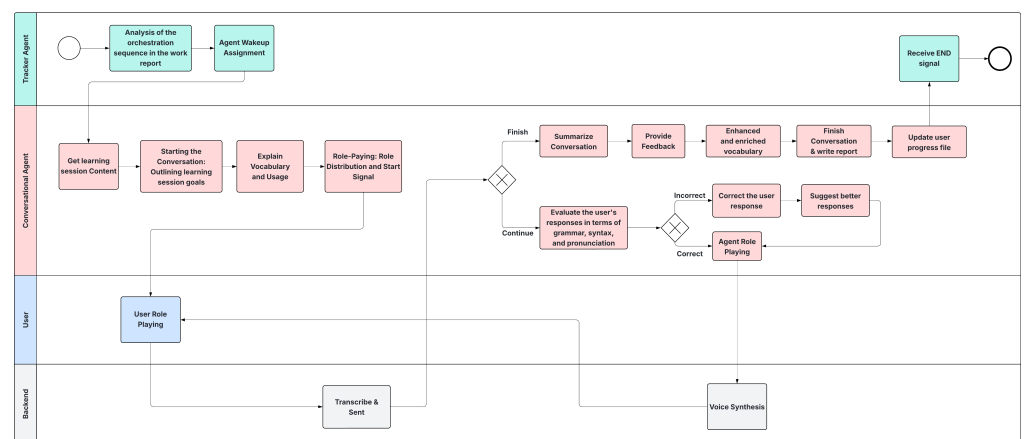


Figure 5. BPMN diagram for the Conversational Tutor Agent. Gateway annotations show decision logic (e.g., G3: ‘>2 errors → review’) for explainability.

4.2. Mapping BPMN to MAS

We translate each BPMN element into a corresponding construct in our LangGraph-based MAS. Table 3 summarizes the mapping from BPMN notation to LangGraph concepts and MAS components.

Table 3. Mapping BPMN Elements to LangGraph Concepts and MAS Components.

BPMN Element	LangGraph Concept	MAS Component	Function
Pool	Agent Node	Agent Class	Encapsulates a high-level role (e.g., Communicator, Orchestrator)
Lane	Tool Node	Agent Capability	Provides an external service or helper (e.g., <code>getFiles</code>)
Task	Task Node	Method Invocation	Executes a concrete operation (e.g., <code>generateRecommendation</code>)
Gateway	Router	Routing Logic	Evaluates conditions and selects outgoing edge
Data Object	State Variable	Memory Store	Holds persistent data (user profile, progress, curriculum)
Message Flow	Message Edge	Inter-Agent Message	Transmits data or control between agents

Each BPMN element is compiled into a LangGraph node or edge, enabling executable MAS.

4.2.1. Agent and Tool Nodes

In the generated LangGraph, each *Pool* becomes an Agent Node with its class definition and personality. Each *Lane* is realized as one or more Tool Nodes attached to the agent, encapsulating external operations (database lookups, API calls, OCR, etc.). For example, the Communicator agent uses a `getFiles` tool node to fetch `UserProfile`, `LatestProgressFile`, and `CurriculumOutline` efficiently.

4.2.2. Routers and Conditional Edges

Gateways in BPMN become Router Nodes in LangGraph that inspect the agent's current state or message payload and dispatch control along the correct outgoing edge. Conditional edges carry predicates (e.g., "user accepted recommendation?") that determine which path the workflow follows at runtime.

4.2.3. Message Passing

Message Flows map directly to Message Edges connecting agent nodes. These edges carry structured payloads, such as the `RAGQuery`, `ContentReport`, and `SequenceReport`, ensuring that each agent receives exactly the information it needs to proceed.

4.2.4. Example: Communicator Routing

Figure 6 shows the Communicator's internal routing logic. After generating a recommendation, the Communicator's router can perform the following:

- Loop back to itself (*Continue*) if the learner requests adjustments.
- Invoke its `communicator_call_tool` node (*Call tool*) to re-fetch profile data.
- Transition to the Orchestrator node (*Go orchestrator*) once the recommendation is approved.

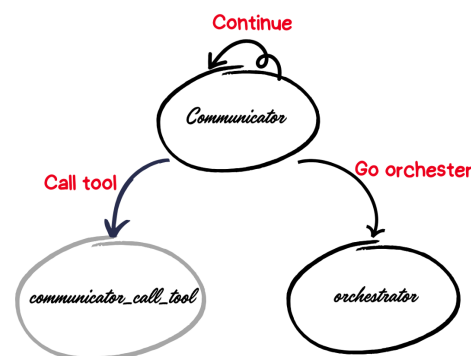


Figure 6. Example of Communicator node routing in LangGraph.

4.2.5. Handling Multiple User Inputs

Unlike simple linear workflows, our language-learning process requires repeated user interactions at various stages. We handle this by embedding input-expectation logic within routers: when a router determines "await user input," it triggers a call to the front end, pauses on an intermediate catch event, and resumes execution once the UI returns the learner's response. This pattern supports multi-step dialogues and ensures smooth, stateful conversations across the entire session.

By compiling BPMN pools, lanes, tasks, gateways, and message flows into LangGraph's nodes, tool nodes, routers, and edges, we obtain an executable, stateful graph representation of our pedagogical workflow, bridging formal process modeling with a robust multi-agent implementation.

4.3. Multi-Agent Architecture

Our MAS architecture consists of specialized agents:

- Communicator Agent: First interface with users, providing personalized recommendations based on learner profiles and progress.
- Orchestrator Agent: Manages workflow, retrieves relevant content, and coordinates agent activation.
- Tracker Agent: Monitors workflow execution and learner progress.
- Tutor Agents: Specialized agents for different learning aspects:
 - Conversational Agent: Facilitates speaking practice.
 - Reading Agent: Guides reading comprehension.
 - Listening Agent: Manages listening exercises.
 - QA Agent: Handles interactive questions.
 - Grammar Summary Agent: Provides grammatical explanations.
- Human Validator: Reviews and approves generated content.

4.4. LangGraph Implementation and Prompt Orchestration

To execute our BPMN-modeled workflows as running LLM agents, we leverage LangGraph, an extension of LangChain [55] that compiles a stateful directed graph from process definitions. Unlike LangChain’s acyclic chains, LangGraph supports cycles and loops, enabling continuous re-evaluation and adaptive, agent-like behaviors in real time.

4.4.1. LangGraph Architecture

A LangGraph program is a *stateful graph* composed of the following:

Nodes: Each node represents a computation phase, often an LLM-driven task executor. Nodes process user inputs, generate or transform text, invoke external tools (e.g., RAG lookups, STT/TTS), and update shared state.

Edges: Unconditional edges define fixed sequences, while *conditional edges* evaluate predicates (e.g., “user accepted recommendation?”) to branch dynamically.

LangGraph thus provides the following:

- Task Looping: Nodes may loop to themselves until a gateway condition is satisfied.
- Conditional Routing: Router nodes inspect state or outputs and select the correct outgoing edge.
- Persistent State Management: Message payloads and node states persist across turns, so each agent “remembers” prior context.

4.4.2. Prompt Engineering for Agent Behavior

Precise prompt construction is essential for controlling each agent:

1. Clarity and Role Definition: “You are the Conversational Agent tasked with...”
2. Stepwise Instructions: Numbered or bullet steps guide the model through its workflow.
3. Contextual Anchoring: Inject RAG-retrieved content chunks to ground responses.
4. Error Handling: Include conditional clauses (e.g., “If the user’s answer is incorrect, provide feedback and re-prompt”).
5. Iterative Refinement: Collect performance metrics after each session and refine prompts to reduce ambiguity and hallucinations.

4.4.3. Integrating Prompts into Nodes

Each BPMN *Task* maps to a LangGraph *TaskNode* with a customized prompt:

```
set_agent_prompt(agent_node, prompt_template, tools=...)
```

For instance, the Conversational Agent is instantiated via the following:

```
create_tutor_agent(
    name="conversational",
    prompt=CONVERSATION_PROMPT,
    tools=[stt_tool, tts_tool]
)
```

where CONVERSATION_PROMPT guides greeting, role-play, feedback, and report-writing steps.

4.4.4. Example Prompt Templates

To make our prompt engineering concrete, we include full templates in Appendix B:

- Listing A1: Communicator Agent system message, showing role definition and basic RAG context setup.
- Listing A2: Conversational Tutor Agent prompt, including:
 - *Role Definition* (“You are a Conversational Agent...”).
 - *RAG Context Injection* (e.g., Thema: “Wéi heescht Dir?”, Kategorie: “Dialogs”, Agent: “Conversational”).
 - *Error-Handling Logic* (e.g., “IF user_error THEN provide corrective feedback and re-prompt”).

4.4.5. Graph Compilation and Execution

Upon startup, LangGraph compiles all agent and tool nodes, routers, and message edges into a single directed graph (Figure 7).

Execution proceeds:

1. The `__start__` node dispatches control to `communicator`.
2. `communicator` interacts with the learner (loop/tool/orchestrator branches).
3. `orchestrator` retrieves RAG content, validates with the teacher, and signals `tracker`.
4. `tracker` sequentially activates each tutor agent (`reader`, `listening`, `questionAnswering`, `grammarSummary`), awaiting each `EndSession`.
5. After all tutor nodes complete, `tracker` issues `__end__`, concluding the session.

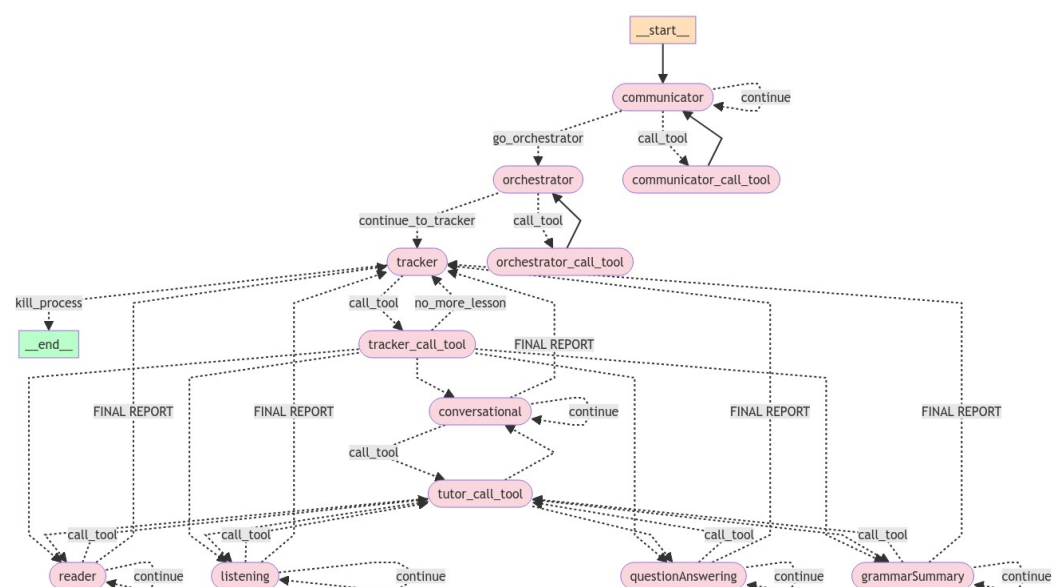


Figure 7. LangGraph-compiled graph for the MSA architecture. Solid arrows denote direct flow; dashed lines denote tool invocations and conditional routers.

This combination of stateful looping, conditional routers, and curated prompt templates ensures our BPMN-designed pedagogy is executed faithfully and transparently by LLM agents.

4.5. Voice Integration: STT and TTS

To support spoken interaction and pronunciation practice, we integrate both STT and TTS pipelines into our multi-agent architecture. These components enable the system to listen to learner utterances and respond with natural audio.

4.5.1. Speech-to-Text (STT)

Developing STT for Luxembourgish faces two main challenges: the language’s low-resource status (scarce transcribed corpora) and the trade-off between monolingual vs. multilingual acoustic models. We evaluated two state-of-the-art pre-trained models on a 100-utterance test set (3–15 s per clip) derived from INL audio materials:

We measure transcription accuracy via Word Error Rate (WER) widely used in pronunciation error detection [56]. (Equation (1)):

$$WER = \frac{S + D + I}{N} \times 100\% \quad (1)$$

where S , D , I are the counts of substitutions, deletions, and insertions, and N is the total words in the reference.

The comparative STT performance on our Luxembourgish test set is shown in Table 4.

Table 4. STT Model Performance on Luxembourgish Test Set.

Model	Pretraining	Fine-Tuning Data	WER
wav2vec2-large-xlsr-53-842h-luxembourgish-14h	Multilingual (53 langs)	842 h unlabeled + 14 h labeled	28%
whisper_large_lb_ZLS_v4_38h	OpenAI Whisper base	14 h → 38 h labeled Luxembourgish	18%

The Whisper model achieves a substantially lower WER, making it our chosen STT backend.

4.5.2. Text-to-Speech (TTS)

High-quality Luxembourgish TTS resources are scarce. We evaluated existing corpora and models before selecting and fine-tuning a multilingual Coqui VITS system.

- Data Source: 1b-de-fr-en-pt-12800-TTS-CORPUS (12,800, WAV samples @16 kHz, 18 speakers, five languages including Luxembourgish).
- Model: 1b-de-fr-en-pt-coqui-vits-tts, a multilingual, multi-speaker VITS model fine-tuned on the above corpus. VITS combines GANs and VAEs in an end-to-end TTS architecture, requiring no external alignments.
- Results and Selection: Fine-tuned Coqui VITS [57] produced natural, intelligible Luxembourgish speech, outperforming MaryTTS-based alternatives. Given its high quality and the lack of superior open models, we adopt Coqui VITS for all agent voice output.

4.5.3. Integration into Multi-Agent Workflow

Both STT and TTS are exposed as *Tool Nodes* in LangGraph. In each tutor agent’s BPMN-derived activity diagram, spoken user input is routed through the STT node, and the agent’s response text is rendered via the TTS node before being played back to the learner. This seamless audio loop underpins pronunciation drills, listening comprehension tasks, and conversational practice across all agents.

4.5.4. STT/TTS Integration

Whisper-based speech recognition and Coqui VITS TTS are tightly integrated into our multi-agent pipeline to provide an immersive voice-enabled learning experience. In our evaluation on a 100-utterance test set derived from INL audio materials, we compared two state-of-the-art ASR models:

- `wav2vec2-large-xlsr-53-842h-luxembourgish-14h`: A multilingual model pre-trained on 53 languages and fine-tuned with 842 h of unlabeled plus 14 h of labeled Luxembourgish speech, which achieved a WER of 28%.
- `whisper_large_1b_ZLS_v4_38h`: OpenAI’s Whisper base model, further fine-tuned on 38 h of labeled Luxembourgish data by the Zentrum für d’Lëtzebuerger Sprooch (ZLS), which achieved a superior WER of 18%.

Given the Whisper [58] model’s substantially lower error rate—especially important for capturing Luxembourgish’s phonological nuances—we selected `whisper_large_1b_ZLS_v4_38h` as our STT backend. For TTS, we fine-tuned the multilingual Coqui VITS model on the `1b-de-fr-en-pt-12800-TTS-CORPUS`, yielding natural, intelligible Luxembourgish speech that outperforms MaryTTS-based alternatives. Embedding these tools as LangGraph tool-nodes ensures each tutor agent can seamlessly listen, evaluate, and speak back to the learner in real time, greatly enhancing both interactivity and pedagogical effectiveness.

5. RAG-Enhanced Knowledge Base

RAG augments LLMs with external knowledge sources to reduce hallucinations and improve factual accuracy. In our Luxembourgish learning platform, RAG grounds every agent’s output in vetted INL textbook content, ensuring pedagogical soundness.

5.1. Why RAG for Low-Resource Languages

Luxembourgish is a classic low-resource language, comprising only $\approx 0.1\%$ of web text compared with $\approx 52\%$ for English. Vanilla LLMs therefore often generate errors or generic responses. RAG addresses these shortcomings by performing the following:

- **Relevance**: Retrieving domain-specific content (INL textbooks) tailored to each learner’s level.
- **Accuracy**: Anchoring generation in factual excerpts, bolstering learner trust.
- **Pedagogical Alignment**: Dynamically selecting material that matches Common European Framework of Reference for Languages (CEFR) aligned chapters and topics.

5.2. RAG Pipeline

Our RAG implementation consists of two phases:

5.2.1. Retrieval

1. **Document Preparation**:
 - Scan INL textbooks (A1–B2) and convert pages to Markdown via GPT-4 Vision OCR [59].
 - Clean and normalize text (remove headers/footers, correct OCR errors).
2. **Chunking and Splitting**: We employ *agentic chunking* to mirror textbook structure:
 - **Splitter Agent**: Divides each topic into semantically coherent “learning blocks.”
 - **Organizer Agent**: Groups blocks by chapter and topic, preserving pedagogical order.
3. **Embedding and Storage**: Each chunk is embedded and stored in ChromaDB [60]. We selected `bge-large-en-v1.5` after benchmarking on MTEB and our pilot RAGAs evaluation as the best trade-off between latency, relevance, and open-source licensing.

5.2.2. Generation

1. Query Embedding and Matching: Learner queries or agent prompts are embedded and matched against stored vectors via cosine similarity to retrieve the top- k chunks.
2. Contextual Response: Retrieved chunks are prepended to the LLM prompt (e.g., GPT-4), which generates the final answer, reflecting both the model's internal knowledge and the verified textbook content.
3. Explainability Tags: Each response includes semantic source metadata drawn from chunk fields: [Source: Thema="Wéi heescht Dir?", Kategorie="Dialogs", Agent="Conversational"] enabling learners and educators to verify content against original materials.

5.3. Embedding Model Selection

We evaluated candidate models on two criteria:

- Latency: Time to embed the full INL corpus (e.g., text-embedding-3-large completed in ≈ 53 s, while others averaged ≈ 3 h).
- Relevance (RAGAs): Performance on context relevancy, faithfulness, and answer relevancy.

Although text-embedding-3-large achieved the fastest embeddings, it underperformed on Luxembourgish relevance. voyage-large-2-instruct scored well but is proprietary. bge-large-en-v1.5 delivered top relevance (context relevancy 0.87, faithfulness 0.82, answer relevancy 0.85) and is open-source. We therefore adopted bge-large-en-v1.5 to balance speed, accuracy, and cost.

5.4. Evaluation with RAGAs

Retrieval-Augmented Generation Assessments (RAGAs) is a reference-free evaluation framework that decomposes RAG performance into retrieval and generation metrics [41].

- Context Relevancy.
- Context Precision.
- Context Recall.
- Faithfulness.
- Answer Relevancy.
- Answer Correctness.

These metrics collectively measure how well the system retrieves pertinent passages and how accurately the model's answers reflect both the retrieved contexts and the learner's query intent.

5.5. Building a Robust Knowledge Base

By combining GPT-4 Vision OCR, agentic chunking, bge-large-en-v1.5 embeddings, and ChromaDB storage, we construct a high-quality vector store of Luxembourgish educational content. This foundation enables our multi-agent system to generate accurate, contextually relevant, and pedagogically aligned learning activities, overcoming the data scarcity that typically hinders low-resource language applications.

6. Implementation and Use Case

We have realized the architecture described above as a web-based prototype specifically for Luxembourgish learning (demo video and additional details in [16]). The system ingests INL A1–B2 materials and provides a full end-to-end learning experience driven by our MAS.

6.1. Technology Stack

- Frontend: React.js, renders the learner dashboard, chat interface, and course navigation, and streams audio via Web Audio API.
- Backend: FastAPI (Python), exposes REST and WebSocket endpoints for user authentication, agent orchestration, and real-time messaging.
- Core Agents: Implemented with LangGraph on top of LangChain, compiles BPMN-derived workflows into a stateful directed graph of TaskNodes and ToolNodes.
- RAG Vector Store: ChromaDB, stores pedagogically chunked INL content; queried via cosine-similarity retrievers.
- STT/TTS: OpenAI Whisper (whisper_large_lb_ZLS_v4_38h) for transcription; Coqui VITS (lb-de-fr-en-pt-coqui-vits-tts) for speech synthesis.

6.2. End-to-End Workflow

When a learner logs in, the frontend invokes the Communicator Agent via FastAPI.

1. The Communicator:
 - Retrieves the user's profile, progress, and curriculum metadata.
 - Constructs and displays a personalized learning path in the React UI.
 - Upon learner approval, emits a `go_orchestrator` event.
2. The Orchestrator Agent then performs the following:
 - Queries ChromaDB for the next topic's content.
 - Sends the raw material to a human teacher for quick validation (teacher-in-the-loop).
 - Builds two reports: (i) validated content for tutor agents and (ii) the ordered list of agent tasks.
 - Emits `continue_to_tracker`.
3. The Tracker Agent:
 - Parses the sequence report and dispatches start signals to each Tutor Agent in turn.
 - Listens for each agent's completion or exit signals.
 - Aggregates intermediate progress and updates the learner's profile.
4. Each Tutor Agent: (Conversational, Reading, Listening, Grammar, Q&A) runs its BPMN-modeled activity diagram as a LangGraph TaskNode:
 - It fetches its specific content from the Orchestrator's report.
 - Interacts with the learner via WebSocket streams (text + STT/TTS audio).
 - Sends real-time feedback and performance metrics back to Tracker.
 - Loops or branches as defined by the BPMN gateways.

6.3. Demonstration Highlights

Rather than reprinting screenshots here, we refer readers to the demonstration paper [16], where you can see the following:

- Learner dashboard flows in React,
- Chat-based dialogues powered by Conversational Agent.
- Listening exercises with real-time transcription.
- Grammar drills and Q&A sessions reflecting adaptive branching.

This fully integrated prototype confirms that our BPMN-to-MAS design, RAG-augmented content, and voice-enabled agents deliver a cohesive, adaptive, and pedagogically sound learning experience for Luxembourgish learners.

7. Evaluation

We evaluated our platform along three dimensions: (i) response accuracy via RAGA metrics, (ii) system effectiveness and learner engagement through a pilot survey, and (iii) overall usability and pedagogical alignment. Given that this study is positioned as a proof-of-concept, the evaluation focuses on immediate indicators relevant to our research questions rather than long-term language proficiency gains. All materials, queries, and survey instruments are documented for reproducibility.

7.1. Response Accuracy with RAGAs

To assess the factual consistency and relevance of our RAG-enhanced agents, we applied the RAGA [41] framework on a held-out set of 200 Luxembourgish queries drawn from INL textbooks. Queries were selected to represent all CEFR A1–B2 skill areas covered in the textbook (reading, listening, grammar, conversation), ensuring balanced coverage across agent roles. Table 5 summarizes the key metrics:

Table 5. RAGAs Evaluation Metrics for our RAG-Enhanced Knowledge Base.

Metric	Score
Context Relevancy	0.87
Faithfulness	0.82
Answer Relevancy	0.85

RAGA metrics are reference-free and evaluate both retrieval quality and generative accuracy.

These results indicate strong contextual grounding (0.87) and high alignment between retrieved passages and generated answers, effectively reducing hallucination compared with a baseline single-agent LLM. This directly supports Research Question 2 by demonstrating measurable improvement in grounding and faithfulness when RAG is embedded in our BPMN-to-MAS workflow.

7.2. System Effectiveness and Learner Experience

We conducted a pilot study with 14 students who used the platform for two 30 min sessions. Participants were students from diverse academic fields, with Luxembourgish proficiency levels ranging from total beginners to native speakers, and mixed native languages (including French, German, and English). Sessions took place in a supervised lab setting to ensure consistent technical conditions. Table 6 presents the aggregated survey responses:

Table 6. Survey Response Summary.

Question	Response Distribution
Ease of Interaction	Very Easy (42.9%), Easy (42.9%), Difficult (14.3%)
Satisfaction with Understanding and Contextual Responses	Satisfied (42.9%), Very Satisfied (28.6%), Neutral (28.6%)
Engagement Level	Very engaging (71.4%), Moderately engaging (28.6%)
Likelihood to Continue	Likely (71.4%), Very Likely (14.3%), Neutral (14.3%)

Survey responses aggregated from 14 Luxembourgish learners in pilot testing.

Key takeaways from this survey include the following:

- Ease of Interaction: 85.8% found the chatbot Very Easy or Easy.
- Satisfaction: 71.5% were Satisfied or Very Satisfied with contextual responses.
- Engagement: 71.4% rated the experience as Very engaging.
- Continued Use: 85.7% are Likely or Very Likely to continue using the system.

Qualitative feedback highlighted the seamless transitions between agents and the usefulness of personalized recommendations. Several learners reported improved confidence

in pronunciation and grammar after interacting with the Conversational and Grammar Summary Agents. This feedback addresses Research Question 1, confirming that the BPMN-structured MAS sequencing was perceived by end-users as both coherent and pedagogically beneficial.

7.3. Usability and Pedagogical Alignment

The platform's React + FastAPI prototype (as detailed in the accompanying demonstration paper [16]) features the following:

- Responsive Interface: Login/dashboard, chat sessions, and progress tracking.
- Agent Workflows: Automatic sequencing of Conversational, Reading, Listening, Q&A, and Grammar agents via BPMN-defined flows.
- STT/TTS Integration: Whisper-based speech recognition (18% WER) and Coqui VITS TTS for immersive voice interaction.

User testing confirmed that the multi-agent orchestration (via LangGraph) maintained a structured, pedagogically sound flow, while the RAG component ensured content accuracy, thereby delivering both high usability and learning effectiveness. The pedagogical alignment observed in this short-term study suggests applicability to other low-resource languages with similar skill-segmented curricula, partially addressing Research Question 3.

7.4. Conclusion of the Evaluation

The combined quantitative and qualitative results demonstrate that our BPMN-based MAS, when enhanced with RAG and robust STT/TTS, offers a reliable, engaging, and pedagogically aligned platform for low-resource language learning. Given the pilot scope and time constraints, this evaluation should be interpreted as an initial validation step. A follow-up longitudinal study is planned to measure sustained proficiency gains and compare performance against baseline single-agent chatbots.

7.5. Limitations

While our system demonstrates promising results, several limitations are worth acknowledging:

1. Model Dependencies: Performance relies on proprietary LLMs (GPT-4) and Whisper STT, limiting control over updates and accessibility for resource-constrained institutions. However, these components can be replaced with open-source alternatives that offer similar capabilities to ChatGPT and Whisper, ensuring the architecture remains usable in fully open-source deployments.
2. Human Validation Bottleneck: Teacher-in-the-loop content approval, while ensuring accuracy, creates scalability challenges for large learner groups.
3. Luxembourgish Specificity: Evaluations focused solely on Luxembourgish; generalizability to other low-resource languages with non-Latin scripts (e.g., Uralic or Bantu languages) remains unverified.
4. Short-Term Engagement Metrics: Pilot studies measured immediate usability but not long-term proficiency gains (e.g., CEFR progression over 6+ months). Additionally, the pilot study's small sample size ($n = 14$) should be increased in future studies.
5. No Control Group: The pilot did not include a control group for comparison against alternative teaching methods or non-MAS language learning tools. This was due to time and resource constraints in the current study design. Future evaluations will incorporate control and experimental groups to enable statistically robust comparisons of learning outcomes.

These limitations stem from the exploratory nature of the work and the scope of available resources during the study period; they will inform the design of subsequent evaluations.

8. Conclusions and Future Work

We have introduced a novel, BPMN-based design methodology for MAS that inherently embeds XAI through workflow transparency and knowledge grounding. BPMN's visual formalism demystifies AI agent behaviors, enabling stakeholders to trace pedagogical decisions (e.g., why a grammar activity followed a listening task), while RAG provides verifiable knowledge provenance that is critical for trust in low-resource contexts. Our methodology integrates RAG-augmented knowledge access, STT/TTS pipelines, and human-in-the-loop validation. Our approach leverages formal process modeling to generate modular, scalable, and pedagogically coherent agent workflows. In a Luxembourgish learning prototype, this architecture achieved high response faithfulness (0.82) and relevance (0.85) under RAGA metrics, reduced hallucinations, and garnered strong learner satisfaction (85.8% ease of use, 71.4% engagement). While the current deployment focuses on Luxembourgish, the architecture is language-agnostic: replacing the knowledge base and adjusting BPMN models enables immediate application to other low-resource or specialized domains. This positions the work as a transferable framework rather than a language-specific solution. The study's pilot-scale evaluation limits conclusions about long-term learning gains, but it provides clear evidence that the integration of BPMN, MAS, and RAG can address the three research questions posed in the introduction.

Future work includes the following:

- **Automate BPMN Generation:** Develop tools to derive BPMN diagrams directly from curriculum specifications or learning objectives, reducing manual modeling effort.
- **Broaden Curriculum Coverage:** Extend our pipeline to additional CEFR levels (C1–C2) and subject domains (e.g., business, technical language).
- **Enhanced Teacher-in-the-Loop:** Introduce richer interfaces and analytics dashboards for instructors to review, adjust, and annotate agent workflows and content.
- **Adaptive Learning Algorithms:** Integrate reinforcement learning and learner modeling to personalize task sequencing dynamically based on real-time performance data.
- **Longitudinal Studies:** Conduct extended field trials across diverse learner populations and languages to evaluate long-term efficacy, retention gains, and transfer to real-world communication.
- **Improve Explainability:** Develop teacher-facing dashboards to visualize BPMN execution logs and RAG source attributions, enhancing real-time explainability. Applying model-agnostic XAI methods could be considered, such as Local Interpretable Model-agnostic Explanations (LIME) for text and SHapley Additive exPlanations (SHAP) for transformers.

This work proposes a blueprint for AI-powered language tutors that uphold pedagogical integrity, support low-resource languages, and adapt seamlessly as educational needs evolve. By formalizing workflows through BPMN and grounding knowledge in RAG, we provide initial evidence that XAI principles (transparency, traceability, and human oversight) can be inherently integrated into complex MAS. As future work, particular emphasis will be placed on quantifying the impact of these XAI mechanisms on learner trust, instructional quality, and knowledge retention across multiple linguistic and cultural contexts. Choosing a low-resource language, Luxembourgish, for the proof of concept was challenging, and the application of other languages is future work.

Author Contributions: Conceptualization, S.N., Y.M. and A.N.; Methodology, H.T., S.N., Y.M. and M.E.F.; Software, H.T. and M.E.F.; Validation, S.N., Y.M., A.N. and A.A.-T.; Formal analysis, H.T. and S.N.; Investigation, H.T., Y.M. and M.E.F.; Resources, M.E.F. and M.D.; Data curation, H.T. and M.E.F.; Writing—original draft, H.T., S.N. and Y.M.; Writing—review & editing, A.N., A.A.-T. and M.D.;

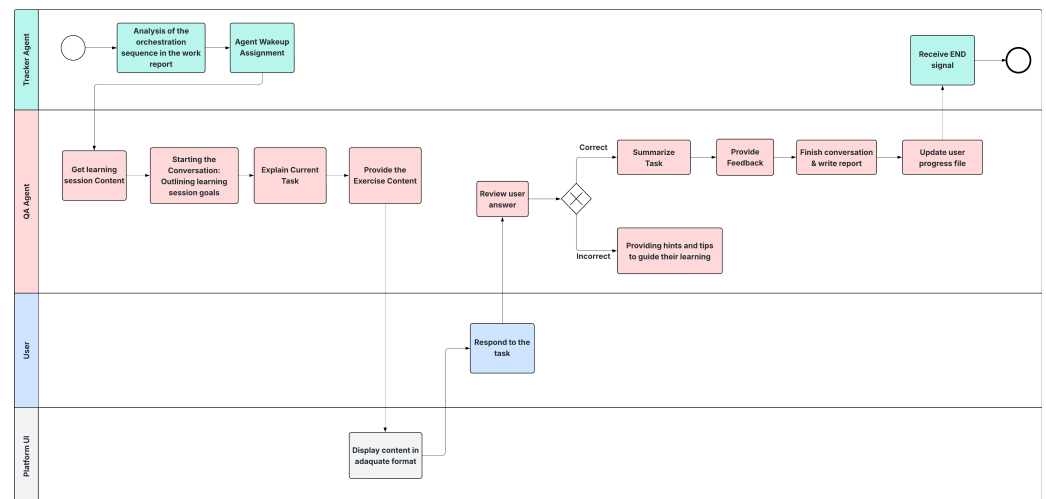


Figure A3. BPMN diagram for the Question Answering Tutor Agent. The gateway annotations illustrate the decision logic (e.g., Q1: if `answer == key`, the flow proceeds as correct; otherwise, the system provides a hint and allows a retry), thereby enhancing explainability.

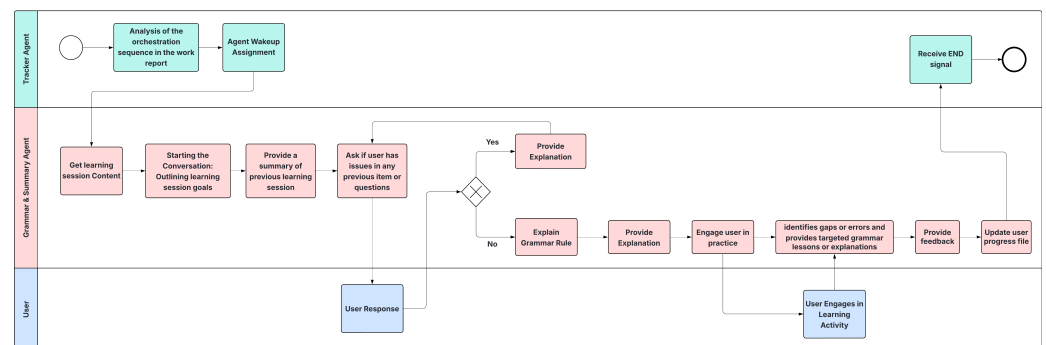


Figure A4. BPMN diagram for the Grammar and Summary Tutor Agent. Gateway annotations show decision logic (e.g., Gs1: '`user_asks_question` → `answer`; else → `practice sentence`') for explainability.

Appendix B. Sample Prompt Templates

Listing A1. Communicator Agent system message.

```
system_message="You are the communicator agent, your job is to
communicate with the user in Luxembourgish to generate
a learning recommendation for them"
```

Listing A2. Conversational Tutor Agent prompt.

```
Conversational_Agent_Prompt = ""
```

Dir sidd en digitalen Tutor, spezialiséiert op Sproochléieren
mat vill Erfahrung, besonnesch an der konversationeller Praxis.
Äert Zil ass et, d'Benotzer duerch effektivt Sproochléieren
mat engem konversationellen Usaz ze féieren.
Follegt dës Instruktiounen fir dës z'erreechen:

1. Léierziler setzen:

- Fänkt un, d'Léierziler ze erklären op Basis vum Inhalt,
deen ofgedeckt gëtt.

2. Wierderbuch an Notzung:

- Bedeelegt Iech un Gespréicher, erkläert de benotzte Wierderbuch a motivéiert de Benotzer nei Wierder ze soen oder se an Sätz ze benotzen.

3. Rollenspill:

- Féiert Rollenspillübungen duerch:
- Definéiert de Fokus vum Gespréich.
- Spezifizéiert Är Roll an d'Roll vum Benotzer.
- Gitt dem Benotzer e Signal fir unzufänken.

4. Evaluatioun a Feedback:

- Evaluéiert d'Äntwerte vum Benotzer grammatesch, syntaktesch an a puncto Aussprooch.
- Wann d'Äntwert korrekt ass, spillt Är Roll.
- Wann d'Äntwert falsch ass, spillt d'Roll vum Tutor, korrigéiert de Benotzer, gitt Hinweise an Tipps, dann spillt Är Roll.

5. Resumé an Nofro:

- Resuméiert d'Gespréich, hebt neie Wierderbuch ervir, an erkläert wéi een en benotzt.
- Frot de Benotzer, ob se méi Beispiller wëllen oder schléit besser Äntwerten a Wierderbuch vir.

6. Feedback ginn:

- Gitt ëmmer Feedback iwwer dat, wat de Benotzer geléiert huet an un wat se schaffe sollten.

7. Fortschrëttsbericht:

- Schreift e Bericht iwwer de Fortschrëtt vum Benotzer:
- Resuméiert, wat se erfollegräich geléiert hunn.
- Hieft Beräicher ervir, un deenen se schaffe müssen.
- Identifizéiert all Schwiriegkeeten, déi se beim Léiere haten.

Huelst Iech e Moment Zäit an schafft methodesch un all Schrëtt, benotzt de bereetgestallten Inhalt als Referenz fir ze léieren an nei Léiermaterialien ze generéieren, a kontrolléiert ëmmer, ob de Benotzer Iech follegt.

" " "

Listing A3. Conversational Tutor Agent prompt translated in english.

You are a digital tutor specializing in language learning with extensive experience, especially in conversational practice. Your goal is to guide users through effective language learning using a conversational approach. Follow these instructions to achieve this:

1. Set Learning Objectives

- Begin by explaining the learning objectives based on the content being covered.

2. Vocabulary and Usage

- Engage the user in conversation, explain the vocabulary you use, and encourage them to produce new words or use them in sentences.

3. Role-Play

- Conduct role-play exercises by:
 - Defining the focus of the dialogue.
 - Specifying your role and the user's role.
 - Giving the user a clear signal to begin.

4. Evaluation and Feedback

- Evaluate the user's responses for grammar, syntax, and pronunciation.
 - If the response is correct, proceed with your next line.
 - If the response is incorrect, adopt the tutor role: correct the user, offer hints and tips, then resume the role-play.

5. Summary and Follow-Up

- Summarize the conversation, highlight new vocabulary, and explain how to use it.
- Ask if the user would like more examples or suggestions for better answers and additional vocabulary.

6. Providing Feedback

- Always give feedback on what the user has learned and what they should focus on next.

7. Progress Report

- Write a brief report on the user's progress:
 - Summarize what they have successfully learned.
 - Highlight areas that need further practice.
 - Identify any difficulties they encountered.

Take your time and work methodically through each step, using the provided content as your reference, generating new learning materials as needed, and always checking that the user is keeping up with you.

References

1. Belda-Medina, J.; Calvo-Ferrer, J.R. Using Chatbots as AI Conversational Partners in Language Learning. *Appl. Sci.* **2022**, *12*, 8427. [\[CrossRef\]](#)
2. Huang, L.; Yu, W.; Ma, W.; Zhong, W.; Feng, Z.; Wang, H.; Chen, Q.; Peng, W.; Feng, X.; Qin, B.; et al. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *arXiv* **2023**, arXiv:2311.05232. [\[CrossRef\]](#)

3. Chen, M.H.; Ye, S.X. Extending repair in peer interaction: A conversation analytic study. *Front. Psychol.* **2022**, *13*, 926842. [CrossRef]
4. Huang, Y.; Qin, Z.; Liu, W. Hallucinations in Large Language Models: Challenges and Mitigation Strategies. In Proceedings of the ACL, Toronto, On, Canada, 9–14 July 2023; pp. 1122–1135.
5. Statistics Portal, Luxembourg. Linguistic Diversity on the Rise. Available online: <https://statistiques.public.lu/en/recensement/diversite-linguistique.html> (accessed on 22 June 2025).
6. Statistics Portal, Luxembourg. Nationalities. Available online: <https://statistiques.public.lu/en/recensement/nationalites.html> (accessed on 22 June 2025).
7. Halder, S.; Meyer, T.; Schmidt, L. Challenges in NLP for Low-Resource Languages: The Case of Luxembourgish. In Proceedings of the LREC, Torino, Italia, 20–25 May 2024; pp. 234–241.
8. Radford, A.; Kim, J.W.; Xu, T.; Brockman, G.; McLeavey, C.; Sutskever, I. Robust Speech Recognition via Large-Scale Weak Supervision. In Proceedings of the 40th International Conference on Machine Learning (PMLR), Honolulu, HI, USA, 23–29 July 2023; Volume 202, pp. 28492–28518.
9. Ju-Zaoravi, Y.; Lee, S.X. Online language learning in participatory culture: Digital pedagogy practices in the post-pandemic era. *Educ. Sci.* **2023**, *13*, 1217. [CrossRef]
10. Xie, Q.; Guo, X. L2 teacher support and positive L2 academic emotions: The mediating role of self-efficacy. *J. Psycholinguist. Res.* **2022**, *51*, 124. [CrossRef]
11. Lothritz, C. NLP De Luxe—Challenges for Natural Language Processing in Luxembourg. Doctoral Thesis, University of Luxembourg, Luxembourg, 2023.
12. Cavojský, M.; Bugár, G.; Kormaník, T.; Hasin, M. Exploring the capabilities and possible applications of large language models for education. In Proceedings of the 2023 21st International Conference on Emerging eLearning Technologies and Applications (ICETA), Stary Smokovec, Slovakia, 26–27 October 2023; pp. 91–98. *IEEE Trans. Educ.* **2025**, *68*, 103–116.
13. Abedi, M.; Alshybani, I.; Shahadat, M.; Murillo, M. Beyond traditional teaching: The potential of large language models and chatbots in graduate engineering education. *arXiv* **2023**, arXiv:2309.13059. [CrossRef]
14. Neumann, A.; Yin, Y.; Sowe, S.K.; Decker, S.; Jarke, M. An LLM-driven chatbot in higher education for databases and information systems. *IEEE Trans. Educ.* **2025**, *68*, 103–116.
15. Van Der Peijl, E.; Najjar, A.; Mualla, Y.; Bourscheid, T.J.; Spinola-Elias, Y.; Karpati, D.; Nouzri, S. Toward XAI & human synergies to explain the history of art: The smart photobooth project. In Proceedings of the International Workshop on Explainable, Transparent Autonomous Agents and Multi-Agent Systems, Virtual Event, 3–7 May 2021; Springer: Berlin/Heidelberg, Germany, 2021; pp. 208–222.
16. Tebourbi, H.; Nouzri, S.; Mualla, Y.; Najjar, A. Personalized Language Learning: A Multi-Agent System Leveraging LLMs for Teaching Luxembourgish. In Proceedings of the AAMAS 2025, Detroit, MI, USA, 19–23 May 2025. Available online: <https://www.ifaamas.org/Proceedings/aamas2025/pdfs/p3032.pdf> (accessed on 13 August 2025).
17. Nouzri, S.; El Fatimi, M.; Guerin, T.; Othmane, M.; Najjar, A. Beyond Chatbots: Enhancing Luxembourgish Language Learning Through Multi-agent Systems and Large Language Model. In Proceedings of PRIMA 2024: Principles and Practice of Multi-Agent Systems, Kyoto, Japan, 18–24 November 2024; Lecture Notes in Computer Science; Arisaka, R., Sanchez-Anguix, V., Stein, S., Aydoğan, R., van der Torre, L., Ito, T., Eds.; Springer: Cham, Switzerland, 2025; Volume 15395. [CrossRef]
18. Object Management Group. Business Process Model and Notation (BPMN). Available online: <https://www.bpmn.org> (accessed on 22 June 2025).
19. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.; Rocktäschel, T.; et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Proceedings of the NeurIPS, Virtual, 6–12 December 2020; pp. 9459–9474.
20. Picard, A.; Mualla, Y.; Gechter, F.; Galland, S. Human-computer interaction and explainability: Intersection and terminology. In Proceedings of the World Conference on Explainable Artificial Intelligence, Lisboa, Portugal, 26–28 July 2023; Springer: Berlin/Heidelberg, Germany, 2023; pp. 214–236.
21. Guidotti, R.; Monreale, A.; Ruggieri, S.; Turini, F.; Giannotti, F.; Pedreschi, D. A survey of methods for explaining black box models. *Acm Comput. Surv. (Csur)* **2019**, *51*, 93.
22. Mualla, Y.; Tchappi, I.; Kampik, T.; Najjar, A.; Calvaresi, D.; Abbas-Turki, A.; Galland, S.; Nicolle, C. The quest of parsimonious XAI: A human-agent architecture for explanation formulation. *Artif. Intell.* **2022**, *302*, 103573. [CrossRef]
23. Hemmer, P.; Schemmer, M.; Vössing, M.; Kühl, N. Human-AI complementarity in hybrid intelligence systems: A structured literature review. In Proceedings of the 25th Pacific Asia Conference on Information Systems (PACIS), Dubai, United Arab Emirates, 12–14 July 2021; Volume 78.
24. Glass, A.; McGuinness, D.L.; Wolverton, M. Toward establishing trust in adaptive agents. In Proceedings of the 13th International Conference on Intelligent User Interfaces, Gran Canaria, Spain, 13–16 January 2008; pp. 227–236.

25. Mualla, Y.; Tchappi, I.H.; Najjar, A.; Kampik, T.; Galland, S.; Nicolle, C. Human-agent explainability: An experimental case study on the filtering of explanations. In Proceedings of the 12th International Conference on Agents and Artificial Intelligence, Valletta, Malta, 22–24 February 2020.
26. Liao, Q.V.; Gruen, D.; Miller, S. Questioning the AI: Informing design practices for explainable AI user experiences. In Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems, Honolulu, HI, USA, 25–30 April 2020; pp. 1–15.
27. Mualla, Y. Explaining the Behavior of Remote Robots to Humans: An Agent-Based Approach. Ph.D. Dissertation, Université Bourgogne Franche-Comté, Besançon, France, 2020. Available online: <https://tel.archives-ouvertes.fr/tel-03162833> (accessed on 22 June 2025)
28. Gunning, D. Explainable Artificial Intelligence (XAI). *Defense Advanced Research Projects Agency (DARPA)*. 2017. Available online: <https://www.darpa.mil/program/explainable-artificial-intelligence> (accessed on 1 July 2025).
29. Biran, O.; Cotton, C. Explanation and justification in machine learning: A survey. In Proceedings of the IJCAI-17 Workshop on Explainable AI (XAI), Melbourne, Australia, 19–25 August 2017; pp. 8–13.
30. Contreras, V.; Marini, N.; Fanda, L.; Manzo, G.; Mualla, Y.; Calbimonte, J.-P.; Schumacher, M.; Calvaresi, D. A desire for extracting propositional rules from neural networks via binarization. *Electronics* **2022**, *11*, 4171. [[CrossRef](#)]
31. D'Mello, S.K.; Graesser, A.C. Intelligent Tutoring Systems: How computers achieve learning gains that rival human tutors. In *Handbook of Educational Psychology*, 4th ed.; Schutz, P.A., Muis, K.R., Eds.; American Psychological Association: Washington, DC, USA, 2023; pp. 603–629.
32. García-López, R.; Smith, J.; Martinez, A. BPMN for Educational Process Modeling: A Systematic Approach. *Comput. Educ.* **2023**, *198*, 104–118.
33. Costa, L.F.; Silva, P. Applying BPMN to Adaptive E-Learning Path Modeling: A Case Study. *Educ. Inf. Technol.* **2023**, *28*, 6543–6561. [[CrossRef](#)]
34. Ait, A.; Cánovas Izquierdo, J.L.; Cabot, J. Towards Modeling Human–Agent Collaborative Workflows: A BPMN Extension. *arXiv* **2024**, arXiv:2412.05958.
35. Bergaoui, N.; Ayachi Ghannouchi, S. A BPM-based approach for ensuring an agile and adaptive learning process. *Smart Learn. Environ.* **2023**, *10*, 40. [[CrossRef](#)]
36. Wooldridge, M.J.; Jennings, N.R. Intelligent Agents: Theory and Practice. *Knowl. Eng. Rev.* **1995**, *10*, 115–152. [[CrossRef](#)]
37. Ivanova, T.; Terzieva, V.; Todorova, K. An Agent-Oriented Architecture For Strategy-Based Personalized E-Learning. In Proceedings of the 2021 Big Data, Knowledge and Control Systems Engineering (BdKCSE), Sofia, Bulgaria, 28–29 October 2021; pp. 1–8.
38. Microsoft Research. AutoGen: Enable Next-Gen Large Language Model Applications. Available online: <https://github.com/microsoft/autogen> (accessed on 1 July 2025).
39. LangChain Inc. LangGraph: Building Stateful Multi-Agent Applications. Available online: <https://langchain.com/langgraph> (accessed on 1 July 2025).
40. Chowdhury, S.P.; Zouhar, V.; Sachan, M. AutoTutor Meets Large Language Models: A Language Model Tutor with Rich Pedagogy and Guardrails. *arXiv* **2024**, arXiv:2402.09216. [[CrossRef](#)]
41. Wang, S.; Liu, Y.; Chen, H. RAG Applications in Educational AI: Reducing Hallucinations and Improving Accuracy. *J. AI Educ.* **2024**, *11*, 156–171.
42. Oche, A.J.; Folashade, A.G.; Ghosal, T.; Biswas, A. A Systematic Review of Key Retrieval-Augmented Generation (RAG) Systems: Progress, Gaps, and Future Directions. *arXiv* **2024**, arXiv:2409.15730.
43. Niu, M.; Li, H.; Shi, J.; Haddadi, H.; Mo, F. Mitigating Hallucinations in Large Language Models via Self-Refinement-Enhanced Knowledge Retrieval. *arXiv* **2024**, arXiv:2408.07061.
44. Rzepka, R.; Araki, K.; Kojima, K. Addressing Hallucinations in Educational AI: A Critical Analysis. *Int. J. AI Educ.* **2023**, *33*, 245–263.
45. Duolingo Inc. Language Learning Platform: Features and Limitations. Available online: <https://www.duolingo.com> (accessed on 22 June 2025).
46. Scarlatos, A.; Liu, N.; Lee, J.; Baraniuk, R.; Lan, A. Training LLM-Based Tutors to Improve Student Learning Outcomes in Dialogues. *arXiv* **2024**, arXiv:2407.01651.
47. Laverne, T.; Urvoy, T.; Yvon, F. NLP Resources for Luxembourgish: Current State and Future Directions. In Proceedings of the LREC, Marseille, France, 20–25 June 2022; pp. 3421–3428.
48. Conneau, A.; Khandelwal, K.; Goyal, N.; Chaudhary, V.; Wenzek, G.; Guzmán, F.; Grave, E.; Ott, M.; Zettlemoyer, L.; Stoyanov, V. Unsupervised Cross-lingual Representation Learning at Scale. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, 5–10 July 2020; pp. 8709–8719.
49. Devlin, J.; Chang, M.-W.; Lee, K.; Toutanova, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of the NAACL-HLT 2019, Minneapolis, MN, USA, 2–7 June 2019; pp. 4171–4186.

50. Plum, A.; Ranasinghe, T.; Purschke, C. Text Generation Models for Luxembourgish with Limited Data: A Balanced Multilingual Strategy. In Proceedings of the 12th Workshop on NLP for Similar Languages, Varieties and Dialects (VarDial 2025), Abu Dhabi, United Arab Emirates, 19 January 2025; Association for Computational Linguistics: Abu Dhabi, United Arab Emirates, 2025; pp. 93–104.
51. Lutgen, A.-M.; Plum, A.; Purschke, C.; Plank, B. Neural Text Normalization for Luxembourgish Using Real-Life Variation Data. In Proceedings of the 12th Workshop on NLP for Similar Languages, Varieties and Dialects (VarDial 2025), Abu Dhabi, United Arab Emirates, 19 January 2025; Association for Computational Linguistics: Abu Dhabi, United Arab Emirates, 2025; pp. 115–127.
52. Gilles, P.; Hosseini-Kivanani, N.; Ayité Hillah, L.E. ASRLUX: Automatic Speech Recognition for the Low-Resource Language Luxembourgish. In Proceedings of the 20th International Congress of Phonetic Sciences (ICPhS 2023), Prague, Czech Republic, 7–11 August 2023; Guarant International: Prague, Czech Republic, 2023.
53. Plum, A.; Döhmer, C.; Milano, E.; Lutgen, A.-M.; Purschke, C. LuxBank: The First Universal Dependency Treebank for Luxembourgish. *arXiv* **2024**, arXiv:2411.04813. [[CrossRef](#)]
54. LangChain Inc. LangSmith: Interactive Tooling for Explainable LLM Workflows. Available online: <https://langchain.com/langsmith> (accessed on 1 July 2025).
55. LangChain Inc. LangChain: Building Applications with LLMs Through Composable Chains and Tools. Available online: <https://www.langchain.com> (accessed on 1 July 2025).
56. Strik, H.; Truong, K.; de Wet, F.; Cucchiari, C. Comparing Different Approaches for Automatic Pronunciation Error Detection. *Speech Commun.* **2019**, *113*, 28–39. [[CrossRef](#)]
57. Coqui Inc. Coqui TTS: VITS-Based Text-to-Speech Models. Available online: <https://coqui.ai> (accessed on 1 July 2025).
58. OpenAI. Whisper: Robust Speech Recognition via Large-Scale Weak Supervision. Available online: <https://openai.com/research/whisper> (accessed on 1 July 2025).
59. OpenAI. GPT-4 with Vision: Multimodal Large Language Models. Available online: <https://platform.openai.com/docs/guides/vision> (accessed on 1 July 2025).
60. Chroma Inc. Chroma: Open-Source Embeddings Database for AI Applications. Available online: <https://www.trychroma.com> (accessed on 1 July 2025).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.